

SVTT-T11: Lý thuyết React JS

Chương 1: Chào mừng đến với React (Welcome to React)

Tổng quan: Định vị React trong hệ sinh thái web và hiểu vai trò của các công cụ nền tảng hỗ trợ.

Các khái niệm trọng tâm

- **Triết lý Thư viện (Library):** Hiểu sự khác biệt giữa thư viện và khung làm việc (framework); tại sao React chọn hướng tiếp cận tập trung vào lớp giao diện.
- **Vai trò của Node.js:** Hiểu tại sao một thư viện chạy trên trình duyệt lại cần môi trường Node.js để biên dịch và đóng gói mã nguồn.
- **Mục tiêu của React Developer Tools:** Hiểu cách công cụ này hỗ trợ quan sát cây thành phần và dòng dữ liệu trong thời gian thực.

Câu hỏi thảo luận

1. Giải thích sự khác biệt giữa Thư viện và Khung làm việc. Tại sao React được gọi là một thư viện?
2. Phân tích vai trò của Node.js trong quy trình phát triển ứng dụng React hiện đại.
3. Mục đích sử dụng của React Developer Tools là gì và nó hỗ trợ gì cho người lập trình?

Chương 2: JavaScript cho React (JavaScript for React)

Tổng quan: Nắm vững các tính năng ngôn ngữ làm nền tảng cho cú pháp và tư duy của React.

Các khái niệm trọng tâm

- **Khai báo biến (Scope & Mutability):** Phân biệt lý thuyết về phạm vi và khả năng thay đổi của const, let, var.
- **Cú pháp Hàm mũi tên (Arrow Functions):** Hiểu cách hàm mũi tên thay đổi cách xử lý ngữ cảnh this so với hàm truyền thống.
- **Kỹ thuật Destructuring & Spread:** Hiểu cách truy xuất và sao chép dữ liệu để duy trì tính bất biến (Immutability).
- **Hệ thống Mô-đun (ES Modules):** Hiểu cơ chế phân tách và kết nối mã nguồn thông qua import và export.

Câu hỏi thảo luận

1. Phân biệt phạm vi hoạt động (scope) và mục đích sử dụng của const, let và var.
2. Giải thích sự khác biệt giữa Hàm mũi tên và hàm truyền thống về mặt cú pháp và cách xử lý từ khóa this.
3. Giải thích mục đích sử dụng của Destructuring và toán tử Spread trong việc quản lý dữ liệu.
4. Trình bày cơ chế hoạt động của import và export trong việc mô-đun hóa ứng dụng.

Chương 3: Lập trình hàm (Functional Programming)

Tổng quan: Tiếp cận triết lý lập trình giúp mã nguồn React trở nên ổn định và dễ dự đoán.

Các khái niệm trọng tâm

- **Declarative (Khai báo) vs Imperative (Mệnh lệnh):** Hiểu sự khác biệt giữa việc mô tả "kết quả mong muốn" và "các bước thực hiện chi tiết".
- **Tính bất biến (Immutability):** Hiểu lý do tại sao không được thay đổi dữ liệu gốc và cách tạo ra bản sao dữ liệu mới.
- **Hàm thuần khiết (Pure Functions):** Hiểu đặc điểm của hàm không gây tác động ngoài lề (Side Effects) và luôn trả về kết quả nhất quán.

- **Cơ chế biến đổi mảng:** Hiểu lý thuyết về các phương thức trả về mảng mới như map, filter, reduce.

Câu hỏi thảo luận

1. Phân biệt tư duy lập trình Khai báo và lập trình Mệnh lệnh. React sử dụng tư duy nào?
2. Tại sao tính bất biến (Immutability) lại là yếu tố sống còn trong lập trình React?
3. Thế nào là một hàm thuần khiết và tại sao nó giúp giảm thiểu lỗi logic?
4. Giải thích mục đích sử dụng của các phương thức map, filter, reduce trong việc xử lý dữ liệu mảng.

Chương 4: Cách React hoạt động và Vòng đời Component

Tổng quan: Khám phá kiến trúc Virtual DOM và các giai đoạn tồn tại của một thành phần giao diện.

Các khái niệm trọng tâm

- **Virtual DOM & Reconciliation:** Hiểu cơ chế so sánh bản vẽ ảo để tối ưu hóa việc cập nhật lên giao diện thật.
- **Component Composition:** Hiểu tư duy lắp ghép các thành phần độc lập thành một cấu trúc ứng dụng hoàn chỉnh.
- **Vòng đời Component (Lifecycle):** Nắm vững lý thuyết về 3 giai đoạn: Mounting (khởi tạo), Updating (cập nhật) và Unmounting (hủy bỏ).

Câu hỏi thảo luận

1. Giải thích cơ chế vận hành của Virtual DOM và quy trình Reconciliation (Sự hòa hợp).
2. Trình bày khái niệm Component Composition và lợi ích của nó trong việc tái sử dụng mã nguồn.
3. Giải thích chi tiết các giai đoạn chính trong vòng đời của một Component (Mounting, Updating, Unmounting).

Chương 5: React với JSX (React with JSX)

Tổng quan: Hiểu về lớp trừu tượng JSX giúp kết nối Logic JavaScript với giao diện người dùng.

Các khái niệm trọng tâm

- **Bản chất của JSX:** Hiểu rằng JSX thực chất là các lời gọi hàm `React.createElement` dưới hình thức giống HTML.
- **Vai trò của Babel:** Hiểu quy trình biên dịch mã nguồn từ JSX về định dạng JavaScript thuần mà trình duyệt có thể thực thi.
- **Ràng buộc của JSX:** Hiểu các quy tắc bắt buộc như thẻ bao ngoài duy nhất (Fragment) và thuộc tính kiểu camelCase.

Câu hỏi thảo luận

1. Bản chất thực sự của JSX là gì? Nó liên quan như thế nào đến `React.createElement`?
2. Tại sao quy trình phát triển React cần đến trình biên dịch Babel?
3. Giải thích mục đích sử dụng của Fragment và lý do các thuộc tính trong JSX tuân theo quy tắc camelCase.

Chương 6: Quản lý trạng thái (React State Management)

Tổng quan: Hiểu cơ chế lưu trữ và truyền tải thông tin xuyên suốt ứng dụng.

Các khái niệm trọng tâm

- **State (Trạng thái):** Hiểu khái niệm dữ liệu nội bộ và tại sao việc cập nhật State lại kích hoạt quá trình render lại giao diện.
- **Props & Dòng dữ liệu một chiều:** Hiểu cơ chế truyền dữ liệu từ cha xuống con và tính chất "chỉ đọc" của Props.
- **Controlled vs Uncontrolled Components:** Phân biệt lý thuyết về việc React kiểm soát dữ liệu biểu mẫu so với việc để trình duyệt tự quản lý.

- **Lý thuyết Context API:** Hiểu cơ chế chia sẻ dữ liệu toàn cục để tránh tình trạng truyền dữ liệu qua nhiều cấp trung gian (Prop Drilling).

Câu hỏi thảo luận

1. State trong React là gì và vai trò của nó trong việc cập nhật giao diện người dùng?
2. Giải thích cơ chế dòng dữ liệu một chiều và tính chất "Read-only" của Props.
3. Phân biệt sự khác nhau về mặt lý thuyết giữa Controlled và Uncontrolled Components.
4. Mục đích sử dụng của Context API là gì và nó giải quyết vấn đề gì trong cấu trúc component?

Chương 7: Tăng cường Component với Hook (Hooks)

Tổng quan: Tìm hiểu các công cụ can thiệp vào logic và tối ưu hiệu năng cho thành phần dạng hàm.

Các khái niệm trọng tâm

- **Quy tắc của Hook (Rules of Hooks):** Hiểu tại sao Hook chỉ được gọi ở cấp cao nhất và lý do đằng sau thứ tự gọi của chúng.
- **useEffect (Side Effect Management):** Hiểu cơ chế kiểm soát các tác vụ ngoài lề và vai trò của mảng phụ thuộc (dependency array).
- **useReducer:** Hiểu mô hình quản lý trạng thái tập trung thông qua các "hành động" (actions).
- **Tối ưu hóa (Memoization):** Hiểu lý thuyết về việc ghi nhớ kết quả (memo, useMemo, useCallback) để tránh render thừa.

Câu hỏi thảo luận

1. Trình bày các quy tắc bắt buộc khi sử dụng Hooks và lý do tại sao phải tuân thủ chúng.
2. Giải thích mục đích sử dụng của useEffect và cách mảng phụ thuộc điều khiển thời điểm thực thi của nó.
3. Khi nào nên sử dụng useReducer thay vì useState về mặt lý thuyết quản lý logic?

4. Giải thích mục đích sử dụng của memo, useMemo và useCallback trong việc tối ưu hóa hiệu suất.

Chương 8: Kết hợp dữ liệu (Incorporating Data)

Các khái niệm trọng tâm

- **Vòng đời của một Request:** Hiểu 4 trạng thái lý thuyết: Idle (ngủ), Loading (đang tải), Success (thành công) và Error (lỗi).
- **Mẫu thiết kế Render Props:** Hiểu cách chia sẻ logic thông qua việc truyền một hàm trả về thành phần giao diện.
- **GraphQL vs REST:** Phân tích sự khác biệt về tư duy truy vấn dữ liệu (lấy đúng thứ cần vs lấy mọi thứ có sẵn).

Câu hỏi thảo luận

1. Trình bày 4 trạng thái của một quá trình yêu cầu dữ liệu (Request) và tầm quan trọng của chúng đối với trải nghiệm người dùng.
2. Giải thích mục đích sử dụng của mẫu thiết kế Render Props.
3. So sánh sự khác biệt về mặt tư duy lấy dữ liệu giữa GraphQL và REST API.

Chương 9: Suspense & Error Boundaries

Các khái niệm trọng tâm

- **Error Boundaries:** Hiểu lý thuyết về việc cô lập sự cố để tránh việc lỗi nhỏ làm hỏng toàn bộ ứng dụng.
- **Code Splitting:** Hiểu lợi ích của việc chia nhỏ mã nguồn đối với thời gian tải trang.
- **Suspense:** Hiểu cơ chế tạm dừng hiển thị khi các thành phần chưa sẵn sàng.

Câu hỏi thảo luận

1. Mục đích sử dụng của Error Boundaries là gì trong việc quản lý lỗi giao diện?
2. Giải thích mục đích của Code Splitting và cách nó cải thiện hiệu suất ứng dụng.
3. Thành phần Suspense được sử dụng nhằm mục tiêu gì trong việc xử lý các thành phần tải chậm?

Chương 10: Kiểm thử và Kiểm tra kiểu (Testing & Typechecking)

Các khái niệm trọng tâm

- **Type Safety (An toàn kiểu):** Hiểu lý do tại sao việc xác định kiểu dữ liệu giúp giảm thiểu lỗi logic từ sớm.
- **Triết lý kiểm thử:** Phân biệt lý thuyết giữa Unit Test (kiểm tra đơn vị) và Integration Test (kiểm tra tích hợp).
- **Linting & Formatting:** Hiểu vai trò của việc thống nhất quy chuẩn viết mã trong dự án chuyên nghiệp.

Câu hỏi thảo luận

1. Tại sao việc kiểm tra kiểu dữ liệu (Typechecking) lại quan trọng trong việc bảo trì ứng dụng?
2. Phân biệt mục tiêu của Unit Test và Integration Test trong phát triển phần mềm.
3. Giải thích vai trò của Linting và Formatting đối với chất lượng mã nguồn.

Chương 11: React Router

Các khái niệm trọng tâm

- **SPA (Single Page Application):** Hiểu cơ chế chuyển đổi nội dung mà không tải lại toàn bộ trang web từ máy chủ.
- **Declarative Routing:** Hiểu việc khai báo các tuyến đường như các thành phần giao diện thông thường.
- **Nested Routes:** Hiểu tư duy định tuyến lồng nhau để xây dựng bố cục (layouts) phức tạp.

- **URL Parameters:** Hiểu cách sử dụng tham số trên đường dẫn để hiển thị dữ liệu động.

Câu hỏi thảo luận

1. SPA là gì và lợi ích của nó so với trang web truyền thống là gì?
2. Giải thích mục đích sử dụng của thẻ <Link> và cách nó khác biệt với thẻ <a> thuần túy.
3. Định tuyến lồng nhau (Nested Routes) được sử dụng để giải quyết bài toán gì?
4. Giải thích mục đích của URL Parameters trong việc xây dựng các trang chi tiết.

Chương 12: React và Máy chủ (React and the Server)

Các khái niệm trọng tâm

- **SSR vs CSR:** Phân tích ưu và nhược điểm giữa việc Render tại máy chủ và Render tại máy khách.
- **Khái niệm Hydration:** Hiểu quy trình gắn kết logic JavaScript vào HTML tĩnh đã nhận từ máy chủ.
- **SSG (Static Site Generation):** Hiểu lý thuyết về việc tạo trang web tại thời điểm xây dựng (build time).

Câu hỏi thảo luận

1. So sánh ưu và nhược điểm của Server-Side Rendering (SSR) và Client-Side Rendering (CSR).
2. Giải thích khái niệm Hydration và vị trí của nó trong quy trình SSR.
3. Mục đích sử dụng của Static Site Generation (SSG) là gì và khi nào nên áp dụng nó?

Phiên bản #1

Được tạo 20 tháng 4 2026 02:53:45 bởi TienPVd

Được cập nhật 20 tháng 4 2026 02:54:56 bởi TienPVd